

프롬프트 하나로 데이터 분석 시키기: LangChain으로 배우는 AI 분석

LangChain - Prompt

김종영



RunnableSequence

- 여러 Runnable 객체들을 순차적으로 연결
- 이전 단계의 출력이 다음 단계의 입력이 되는 직렬 파이프라인을 생성
- 가장 일반적인 체인 구성 방식

```
from langchain_core.runnables import RunnableSequence
from langchain_core.prompts import ChatPromptTemplate
from langchain_community.llms import OpenAI

prompt = ChatPromptTemplate.from_template("다음 내용을 요약해줘: {input}")
llm = OpenAI()

# 파이프(|) 연산자를 사용하여 순차 체인 구성
chain = prompt | llm

# invoke 호출 시 순서대로 실행
chain.invoke({"input": "복잡한 글"})
```

RunnableParallel (RunnableMap)

- 여러 Runnable 객체들을 병렬로 실행, 각 실행 결과를 딕셔너리 형태로 리턴
- RunnableMap은 별칭

```
from langchain_core.runnables import RunnableParallel
from langchain_core.prompts import ChatPromptTemplate

# 두 개의 독립적인 Runnable 정의
joke_chain = ChatPromptTemplate.from_template("유머를 만들어줘: {topic}")
story_chain = ChatPromptTemplate.from_template("스토리를 만들어줘: {topic}")

# 딕셔너리 형태로 병렬 체인 구성
parallel_chain = RunnableParallel(
    joke=joke_chain,
    story=story_chain
)

# invoke 호출 시 두 체인이 동시에 실행되고, 결과가 딕셔너리로 반환됨
result = parallel_chain.invoke({"topic": "개미"})
# 예시 결과: {"joke": "개미가...", "story": "옛날 옛적에 개미가..."}
```

RunnablePassthrough

- 입력값을 수정하지 않고 그대로 다음 Runnable로 전달하는 역할
- `.assign()` 메서드를 사용하여 입력 값에 새로운 키와 값을 추가

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.prompts import ChatPromptTemplate

# Passthrough를 사용하여 입력값을 그대로 전달
chain_A = RunnablePassthrough()
chain_A = chain_A | ChatPromptTemplate.from_template("입력값은 {input}입니다.")

result = chain_A.invoke({"input": "Hello"})
print(result)
```

RunnableLambda

- 파이썬 함수를 Runnable 객체로 변환하여 체인 내에서 사용
- 복잡하거나 커스텀된 로직을 파이프라인에 손쉽게 통합

```
from langchain_core.runnables import RunnableLambda

# 글자 수를 세는 람다 함수를 Runnable로 변환
length_checker = RunnableLambda(lambda x: len(x))

# 체인에 포함하여 사용
result = length_checker.invoke("안녕하세요") # 5 반환
```


RunnableBranch

- 특정 조건에 따라 다른 Runnable 체인을 실행하는 조건부 분기

```
from langchain_core.runnables import RunnableBranch
from langchain_core.prompts import ChatPromptTemplate

# 조건 분기될 체인 정의
positive_chain = ChatPromptTemplate.from_template("긍정적인 답변: {input}")
negative_chain = ChatPromptTemplate.from_template("부정적인 답변: {input}")
neutral_chain = ChatPromptTemplate.from_template("중립적인 답변: {input}")

# RunnableBranch를 사용하여 조건에 따라 다른 체인을 실행
branch = RunnableBranch(
    (lambda x: "긍정" in x["input"], positive_chain), # 입력에 '긍정'이 있으면
    (lambda x: "부정" in x["input"], negative_chain), # 입력에 '부정'이 있으면
    neutral_chain # 둘 다 아니면
)

result_a = branch.invoke({"input": "오늘은 긍정적인 하루예요."})
result_b = branch.invoke({"input": "기분이 부정적이에요."})
```

RunnableWithFallbacks

- 기본 Runnable이 실패할 경우, 미리 정의된 대체 (fallback) Runnable을 실행하도록 설정

```
from langchain_core.runnables import RunnableLambda
from langchain_core.runnables.base import RunnableWithFallbacks

# 일부러 에러를 발생시키는 함수
def unreliable_func(x):
    raise ValueError("고의적인 실패")

# 대체로 사용할 함수
def fallback_func(x):
    return "백업 성공: " + x

# 기본 Runnable과 대체 Runnable을 정의
main_runnable = RunnableLambda(unreliable_func)
fallback_runnable = RunnableLambda(fallback_func)

# with_fallbacks를 사용하여 대체 설정
safe_runnable = main_runnable.with_fallbacks([fallback_runnable])

# invoke 호출 시 기본 runnable이 실패하고 fallback runnable이 실행됨
result = safe_runnable.invoke("실패 테스트")
# 예시 결과: "백업 성공: 실패 테스트"
```

